

Lab 2: Docker Containerization Lab Report

Nabin Ranabhat (078BCT050)

February 20, 2025

1 Introduction

Containerization represents a modern approach to virtualization that encapsulates applications alongside their dependencies, enabling consistent execution across diverse computing environments. Docker has emerged as the predominant platform for container management, streamlining the processes of creating, deploying, and orchestrating containers. Unlike conventional virtual machines, containers operate by sharing the host system's kernel, resulting in enhanced performance and reduced resource consumption.

2 Objectives

- Configure Docker environment on Ubuntu operating system
- Deploy and manage a basic web server container
- Develop and build a customized Docker image
- Orchestrate a multi-service application using containers
- Explore and validate container networking principles

3 Tools and Technologies Used

- Ubuntu Linux (Host Operating System)
- Docker Engine (Container Runtime)

- Docker Compose (Multi-container Orchestration)
- Node.js (Backend Runtime Environment)
- Nginx (Web Server)
- MySQL (Database Management System)

4 Implementation Details

4.1 Docker Installation Procedure

The Docker platform was installed utilizing Ubuntu's package management system. Following installation, the Docker daemon was initialized and configured for automatic startup during system boot.

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
docker --version
```

4.2 Deploying a Basic Web Server

A lightweight Nginx container was instantiated using the following command:

```
docker run -d -p 8080:80 nginx
```

The `-d` flag executes the container in detached mode, while `-p` establishes port mapping between the host machine and container environment.

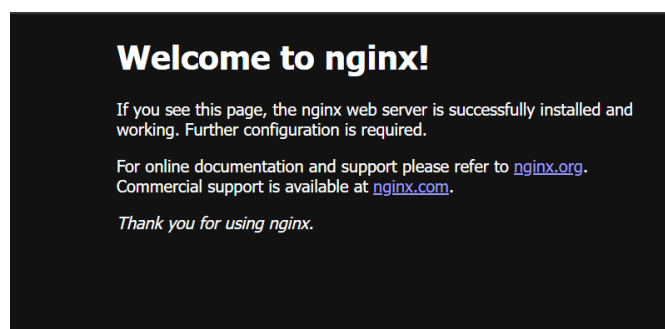


Figure 1: Active Nginx container instance

4.3 Building a Custom Docker Image

Server Implementation

```
const http = require("http");
http.createServer((req,res)=>{
  res.end("Hello Docker");
}).listen(3000);
```

Dockerfile Configuration

```
FROM node:20-alpine
WORKDIR /app
COPY . .
EXPOSE 3000
CMD ["node","server.js"]
```

Image Build and Container Launch

```
docker build -t myapp .
docker run -p 3000:3000 myapp
```

Dockerfile Directive Breakdown

Directive	Function
FROM	Defines base image
WORKDIR	Establishes working directory
COPY	Transfers files to container
EXPOSE	Designates network port
CMD	Specifies startup command

```
anil@DESKTOP-H7MHREK:~$ cd myapp/
anil@DESKTOP-H7MHREK:~/myapp$ docker build -t mynodeapp .
[+] Building 2.5s (8/8) FINISHED
=> [internal] load build definition from dockerfile
=> => transferring dockerfile: 119B
=> [internal] load metadata for docker.io/library/node:20-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/node:20-alpine@sha256:09e2b3d9726018aecf269bd35325f46bf75046a643a66d28360ec71132750ec8
=> => resolve docker.io/library/node:20-alpine@sha256:09e2b3d9726018aecf269bd35325f46bf75046a643a66d28360ec71132750ec8
=> [internal] load build context
=> => transferring context: 59B
=> CACHED [2/3] WORKDIR /app
=> CACHED [3/3] COPY . .
=> => exporting image
=> => exporting layers
=> => exporting manifest sha256:6f8d29ea95e0a94e3b57e21db498dc1f8e0dd06f96d04d57353a7c826ab83dcf
=> => exporting config sha256:dd8b02ea8c3c1f89af971685437eba9eb961fcdc378ef1cf6b5672bdf0ba9093
=> => exporting attestation manifest sha256:c84e4351e2897175442875b32f4666ebe6c314a86d29214a8aea678d4f479c78
=> => exporting manifest list sha256:2e96cf4eb630991161585dc146cad7d42d25717031c00da0ed761bb9a6168ad1
=> => naming to docker.io/library/mynodeapp:latest
=> => unpacking to docker.io/library/mynodeapp:latest
anil@DESKTOP-H7MHREK:~/myapp$
```

Figure 2: Docker image build process

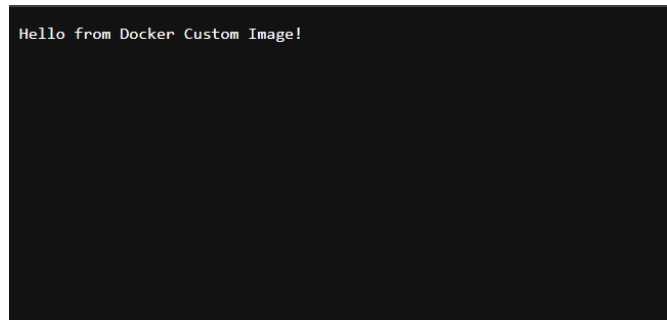


Figure 3: Custom Docker image creation

4.4 Multi-Container Application Deployment

A comprehensive multi-service architecture was implemented incorporating front-end presentation, backend logic, and database persistence layers.

backend/server.js

```
const express=require("express");
const mysql=require("mysql2");
const app=express();

const db=mysql.createConnection({
  host:"db",
  user:"root",
  password:"root",
  database:"testdb"
});

app.get("/",(req,res)=>{
  db.query("SELECT 1",(err)=>{
    if(err) return res.send("DB Error");
    res.send("Backend Connected");
  });
});

app.listen(5000);
```

Backend Dockerfile Configuration

```
FROM node:20-alpine
WORKDIR /app
COPY . .
```

```
RUN npm init -y && npm install express mysql2
EXPOSE 5000
CMD ["node","server.js"]
```

frontend/index.html

```
<h1>Frontend Running</h1>
```

Frontend Dockerfile Configuration

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
```

Docker Compose Orchestration File

```
version: "3.9"
services:

  frontend:
    build: ./frontend
    ports:
      - "8080:80"

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    depends_on:
      - db

  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: testdb
    ports:
      - "3307:3306"
```

Application Initialization

```
docker compose up --build
```

Container-to-container communication was achieved through service name resolution as defined in the compose configuration.

```
root@b8c797c9:~/project# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
8fcb1d60388c   project-frontend  "/docker-entrypoint.sh"  8 seconds ago  Up 7 seconds  0.0.0.0:8080->80/tcp             project-frontend-1
ba2d85c298a4   mysql:5.7       "docker-entrypoint.sh"  2 minutes ago  Up 2 minutes  0.0.0.0:3306->3306/tcp           frontend_mysql
42d18098d3a0   mysql:5.7       "docker-entrypoint.sh"  43 minutes ago  Up 7 seconds  33060/tcp, 0.0.0.0:3307->3306/tcp  project-db-1
root@b8c797c9:~/project#
```

Figure 4: Active multi-container environment



Figure 5: Backend service verification



Figure 6: Frontend service verification

5 Findings and Observations

- Container isolation effectively separated application components

- Consistent application behavior was observed across deployment stages
- Deployment time was significantly reduced compared to traditional methods
- Inter-container networking functioned as expected through service discovery

6 Technical Challenges Encountered

Issue	Root Cause	Resolution
Port allocation conflict	Existing service using port	Modified port mapping configuration
Database connectivity failure	Incorrect host specification	Implemented service name resolution
Insufficient permissions	User not in docker group	Added user to docker group

7 Conclusion

This practical exercise successfully demonstrated the complete Docker workflow including platform installation, container deployment, custom image creation, and multi-service orchestration. The experience confirmed that Docker significantly streamlines application deployment while maintaining environmental consistency. These capabilities make containerization indispensable for contemporary software development practices and cloud-native architecture implementations.